Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

Звіт
До лабораторної роботи №7
Варіант 9
На тему
“functools та operator”

Виконав:
Студент групи ФЕП-22с
Обиход І. В.
Перевірив:
Ас. Остріков Д. А.

Львів 2026

Мета роботи:

- застосувати partial, reduce, singledispatch, total_ordering;
- використати itemgetter, attrgetter, methodcaller замість lambda;
- побудувати конвеєр через часткове застосування;
- створити поліморфний API через singledispatch.

Теоретичні відомості:

- functools.partial фіксує частину аргументів і повертає функцію з меншим інтерфейсом.
- functools.reduce згортає колекцію; з його допомогою реалізується pipeline(value, *funcs).
- singledispatch вибирає реалізацію функції за типом першого аргументу.
- total_ordering добудовує всі порівняння за __eq__ і одним із __lt__/__le__/__gt__/__ge__; operator дає швидкі гетери itemgetter, attrgetter і methodcaller.

Завдання (варіант 9 - Фільми та рейтинги):

Фільми: title, genre, year, rating, votes. Відібрати фільми за роком і мінімальним рейтингом, сформувати Top-N. partial для фільтрів; load_movies через singledispatch; Movie за rating, votes, title.

Хід роботи

У core.py реалізовано pipeline через functools.reduce, хелпери mapf і filt через partial(map, ...) та partial(filter, ...), нормалізацію записів через itemgetter і methodcaller (strip, title, lower). Фільтри year_at_least, rating_at_least і genre_is налаштовуються через partial.

clean_and_rank завантажує дані, нормалізує, фільтрує, сортує за rating і votes спадно та title зростанням і повертає Top-N. Багатокритеріальне сортування з різними напрямками зроблено двома стабільними проходами sorted з itemgetter.

У loaders.py поліморфна функція load_movies на @singledispatch підтримує JSON-рядок, шлях str і pathlib.Path до файлів .json та .csv, відкритий текстовий файл і готовий список словників. Для непідтримуваних типів виникає TypeError.

У models.py клас Movie з @total_ordering реалізує __eq__ і __lt__ за ключем (rating, votes, title), повертає NotImplemented для чужих типів і має узгоджений __hash__. Додатково total_votes і best_by_genre побудовано на reduce, а titles на attrgetter.

Тести перевіряють pipeline, partial, operator, багатокритеріальне сортування, однаковий результат для всіх типів джерел, помилки завантаження та всі оператори порівняння.

Файл core.py

core.py рядки 1-62

```python
"""Конвеєр ЛР7, варіант 9: partial + reduce + operator над фільмами."""

from collections.abc import Callable, Iterable, Mapping
from functools import partial, reduce
from itertools import islice
from operator import add, attrgetter, itemgetter, methodcaller
from typing import Any

from loaders import load_movies
from models import Movie, MovieRecord

get_title = itemgetter("title")
get_genre = itemgetter("genre")
get_year = itemgetter("year")
get_rating = itemgetter("rating")
get_votes = itemgetter("votes")
strip = methodcaller("strip")
to_title = methodcaller("title")
to_lower = methodcaller("lower")


def pipeline(value: Any, *funcs: Callable[[Any], Any]) -> Any:
    """Пропускає значення через функції зліва направо (functools.reduce)."""
    return reduce(lambda acc, f: f(acc), funcs, value)


def mapf(fn: Callable[[Any], Any]) -> Callable[[Iterable[Any]], Iterable[Any]]:
    """map із зафіксованою функцією (partial)."""
    return partial(map, fn)


def filt(pred: Callable[[Any], bool]) -> Callable[[Iterable[Any]], Iterable[Any]]:
    """filter із зафіксованим предикатом (partial)."""
    return partial(filter, pred)


def normalize(row: Mapping[str, object]) -> MovieRecord:
    """Очищує рядки та приводить типи (itemgetter + methodcaller)."""
    return {
        "title": to_title(strip(str(get_title(row)))),
        "genre": to_lower(strip(str(get_genre(row)))),
        "year": int(str(get_year(row))),
        "rating": float(str(get_rating(row))),
        "votes": int(str(get_votes(row))),
    }


def year_at_least(min_year: int, movie: MovieRecord) -> bool:
    """Фільм не старший за min_year."""
    return movie["year"] >= min_year


def rating_at_least(min_rating: float, movie: MovieRecord) -> bool:
    """Рейтинг не нижчий за min_rating."""
    return movie["rating"] >= min_rating


def genre_is(genre: str, movie: MovieRecord) -> bool:
    """Фільм належить жанру."""
    return movie["genre"] == genre.lower()
```

core.py рядки 63-117

```python
def sort_rank(records: Iterable[MovieRecord]) -> list[MovieRecord]:
    """Сортує: rating спадно, votes спадно, title зростанням (стабільний sorted)."""
    by_title = sorted(records, key=get_title)
    return sorted(by_title, key=itemgetter("rating", "votes"), reverse=True)


def top_n(n: int) -> Callable[[Iterable[MovieRecord]], list[MovieRecord]]:
    """Бере перші n записів."""
    return lambda records: list(islice(records, max(n, 0)))


def clean_and_rank(
    src: object,
    top: int = 3,
    *,
    min_year: int = 2000,
    min_rating: float = 7.0,
    genre: str | None = None,
) -> list[MovieRecord]:
    """Завантажує, нормалізує, фільтрує, сортує і повертає Top-N фільмів."""
    filters = [
        filt(partial(year_at_least, min_year)),
        filt(partial(rating_at_least, min_rating)),
    ]
    if genre is not None:
        filters.append(filt(partial(genre_is, genre)))
    result: list[MovieRecord] = pipeline(
        src, load_movies, mapf(normalize), *filters, sort_rank, top_n(top)
    )
    return result


def to_movies(records: Iterable[MovieRecord]) -> list[Movie]:
    """Перетворює записи на об'єкти Movie."""
    return list(map(Movie.from_record, records))


def total_votes(records: Iterable[MovieRecord]) -> int:
    """Сумарна кількість голосів через reduce."""
    return reduce(add, map(get_votes, records), 0)


def best_by_genre(movies: Iterable[Movie]) -> dict[str, Movie]:
    """Найкращий фільм кожного жанру (reduce + порівняння total_ordering)."""
    get = attrgetter("genre")
    return reduce(
        lambda acc, m: {**acc, get(m): max(acc.get(get(m), m), m)},
        movies,
        dict[str, Movie](),
    )


def titles(movies: Iterable[Movie]) -> list[str]:
    """Назви фільмів через attrgetter."""
    return list(map(attrgetter("title"), movies))
```

Файл loaders.py

loaders.py рядки 1-62

```python
"""Поліморфне завантаження фільмів через @singledispatch (ЛР7, варіант 9).

Підтримуються: JSON-рядок, шлях str/Path до .json або .csv, відкритий
текстовий файл (JSON або CSV) і готовий список словників.
"""

import csv
import io
import json
from functools import singledispatch
from pathlib import Path
from typing import Any

Row = dict[str, Any]


def _from_json_text(text: str) -> list[Row]:
    """Розбирає JSON-масив об'єктів."""
    data = json.loads(text)
    if not isinstance(data, list):
        raise ValueError("JSON має містити список фільмів")
    return load_movies(data)


def _from_csv_text(text: str) -> list[Row]:
    """Розбирає CSV із заголовком."""
    return [dict(row) for row in csv.DictReader(io.StringIO(text))]


def _from_text(text: str) -> list[Row]:
    """Визначає формат вмісту: JSON починається з '[' або '{', інакше CSV."""
    stripped = text.strip()
    if stripped.startswith(("[", "{")):
        return _from_json_text(stripped)
    return _from_csv_text(text)


@singledispatch
def load_movies(src: object) -> list[Row]:
    """Повертає список сирих записів фільмів з різних джерел."""
    raise TypeError(f"Непідтримуваний тип джерела: {type(src)!r}")


@load_movies.register
def _(src: str) -> list[Row]:
    """JSON-рядок або шлях до файлу .json/.csv."""
    stripped = src.strip()
    if stripped.startswith(("[", "{")):
        return _from_json_text(stripped)
    return load_movies(Path(stripped))


@load_movies.register
def _(src: Path) -> list[Row]:
    """Файл .json або .csv."""
    text = src.read_text(encoding="utf-8")
    if src.suffix.lower() == ".json":
        return _from_json_text(text)
    if src.suffix.lower() == ".csv":
        return _from_csv_text(text)
    raise ValueError(f"Непідтримуване розширення: {src.suffix}")
```

loaders.py рядки 63-75

```python
@load_movies.register
def _(src: io.TextIOBase) -> list[Row]:
    """Відкритий текстовий файл з JSON або CSV."""
    return _from_text(src.read())


@load_movies.register
def _(src: list) -> list[Row]:  # type: ignore[type-arg]
    """Готовий список словників (повертається копія)."""
    if not all(isinstance(item, dict) for item in src):
        raise ValueError("Очікується список словників")
    return [dict(item) for item in src]
```

Файл models.py

models.py рядки 1-55

```python
"""Моделі ЛР7, варіант 9: нормалізований запис фільму та клас Movie."""

from dataclasses import dataclass
from functools import total_ordering
from typing import TypedDict


class MovieRecord(TypedDict):
    """Нормалізований запис фільму."""

    title: str
    genre: str
    year: int
    rating: float
    votes: int


@total_ordering
@dataclass(frozen=True, eq=False)
class Movie:
    """Фільм із порядком за rating, потім votes, потім title.

    Достатньо __eq__ та __lt__, решту порівнянь добудовує @total_ordering.
    """

    title: str
    genre: str
    year: int
    rating: float
    votes: int

    def _key(self) -> tuple[float, int, str]:
        """Ключ порівняння."""
        return self.rating, self.votes, self.title

    def __eq__(self, other: object) -> bool:
        """Рівність за ключем; для чужих типів - NotImplemented."""
        if not isinstance(other, Movie):
            return NotImplemented
        return self._key() == other._key()

    def __lt__(self, other: object) -> bool:
        """Менше за ключем; для чужих типів - NotImplemented."""
        if not isinstance(other, Movie):
            return NotImplemented
        return self._key() < other._key()

    def __hash__(self) -> int:
        """Хеш узгоджений з __eq__."""
        return hash(self._key())

    @classmethod
    def from_record(cls, record: MovieRecord) -> "Movie":
        """Створює Movie з нормалізованого запису."""
        return cls(**record)
```

Файл main.py

main.py рядки 1-51

```python
"""Точка входу лабораторної роботи №7, варіант 9."""

from pathlib import Path

from core import (
    best_by_genre,
    clean_and_rank,
    mapf,
    normalize,
    pipeline,
    to_movies,
    total_votes,
)
from loaders import load_movies

DATA_DIR = Path(__file__).parent


def run() -> None:
    """Демонструє конвеєр, singledispatch і total_ordering."""
    json_path = DATA_DIR / "sample_data.json"
    csv_path = DATA_DIR / "sample_data.csv"

    print("Top-3 фільми з 2000 року, рейтинг від 7.0 (JSON):")
    for m in clean_and_rank(json_path, top=3):
        print(f"  {m['title']:<20} {m['year']}  {m['rating']}  {m['votes']:>9}")

    print("\nTop-2 sci-fi (CSV, шлях рядком):")
    for m in clean_and_rank(str(csv_path), top=2, genre="Sci-Fi"):
        print(f"  {m['title']:<20} {m['rating']}")

    same = clean_and_rank(json_path, top=5) == clean_and_rank(csv_path, top=5)
    print(f"\nJSON і CSV дають однаковий результат: {same}")

    records = pipeline(json_path, load_movies, mapf(normalize), list)
    movies = to_movies(records)
    print(f"Сумарно голосів: {total_votes(records):,}")
    print(f"Найкращий фільм (max за total_ordering): {max(movies).title}")
    for genre, movie in sorted(best_by_genre(movies).items()):
        print(f"  найкращий у жанрі {genre:<8}: {movie.title}")
    inception, interstellar = movies[0], movies[1]
    print(f"Inception > Interstellar: {inception > interstellar}")


def main() -> None:
    """Запуск демонстрації functools та operator."""
    run()


if __name__ == "__main__":
    main()
```

Файл tests/test_lab07.py

test_lab07.py рядки 1-58

```python
"""pytest-тести ЛР7, варіант 9."""

import io
import json
from functools import partial, reduce
from operator import itemgetter, mul
from pathlib import Path

import pytest

from core import (
    best_by_genre,
    clean_and_rank,
    filt,
    genre_is,
    mapf,
    normalize,
    pipeline,
    rating_at_least,
    sort_rank,
    titles,
    to_movies,
    top_n,
    total_votes,
    year_at_least,
)
from loaders import load_movies
from models import Movie, MovieRecord

ROWS = [
    {
        "title": " the matrix ",
        "genre": "Sci-Fi",
        "year": 1999,
        "rating": 8.7,
        "votes": 10,
    },
    {"title": "inception", "genre": "sci-fi", "year": 2010, "rating": 8.8, "votes": 30},
    {"title": "joker", "genre": "Drama", "year": 2019, "rating": 8.4, "votes": 20},
    {"title": "alpha", "genre": "Drama", "year": 2018, "rating": 8.4, "votes": 20},
    {"title": "cats", "genre": "Musical", "year": 2019, "rating": 2.8, "votes": 5},
]
CSV_TEXT = "title,genre,year,rating,votes\n" + "\n".join(
    f"{r['title']},{r['genre']},{r['year']},{r['rating']},{r['votes']}" for r in ROWS
)


def test_pipeline_order_and_empty() -> None:
    """Тест: pipeline застосовує функції зліва направо."""
    assert pipeline(3, lambda x: x + 1, lambda x: x * 2, lambda x: x + 1) == 9
    assert pipeline(5) == 5


def test_reduce_initial_value() -> None:
    """Тест: reduce з початковим значенням на порожній послідовності."""
    assert reduce(mul, [], 1) == 1
```

test_lab07.py рядки 59-116

```python
def test_partial_filters() -> None:
    """Тест: partial фіксує параметри фільтрів."""
    records = [normalize(r) for r in ROWS]
    recent = list(filt(partial(year_at_least, 2010))(records))
    assert [r["title"] for r in recent] == ["Inception", "Joker", "Alpha", "Cats"]
    good = list(filt(partial(rating_at_least, 8.5))(records))
    assert [r["title"] for r in good] == ["The Matrix", "Inception"]
    drama = list(filt(partial(genre_is, "DRAMA"))(records))
    assert len(drama) == 2


def test_normalize_with_operator() -> None:
    """Тест: нормалізація рядків і типів через itemgetter/methodcaller."""
    row = {
        "title": "  the matrix ",
        "genre": " Sci-Fi",
        "year": "1999",
        "rating": "8.7",
        "votes": "10",
    }
    assert normalize(row) == {
        "title": "The Matrix",
        "genre": "sci-fi",
        "year": 1999,
        "rating": 8.7,
        "votes": 10,
    }


def test_sort_rank_multi_key() -> None:
    """Тест: rating спадно, votes спадно, title зростанням."""
    ranked = sort_rank(map(normalize, ROWS))
    assert [r["title"] for r in ranked] == [
        "Inception",
        "The Matrix",
        "Alpha",
        "Joker",
        "Cats",
    ]


def test_itemgetter_multiple_keys() -> None:
    """Тест: itemgetter для кількох ключів."""
    rows = [{"a": 1, "b": 2}, {"a": 1, "b": 1}, {"a": 0, "b": 9}]
    assert sorted(rows, key=itemgetter("a", "b")) == [
        {"a": 0, "b": 9},
        {"a": 1, "b": 1},
        {"a": 1, "b": 2},
    ]


def test_clean_and_rank_top() -> None:
    """Тест: повний конвеєр з Top-N."""
    result = clean_and_rank(ROWS, top=2, min_year=2000, min_rating=8.0)
    assert [r["title"] for r in result] == ["Inception", "Alpha"]
    assert clean_and_rank(ROWS, top=0) == []
```

test_lab07.py рядки 117-176

```python
def test_singledispatch_all_sources(tmp_path: Path) -> None:
    """Тест: усі типи джерел (рядок, шлях, Path, файл, список) дають один результат."""
    json_file = tmp_path / "movies.json"
    csv_file = tmp_path / "movies.csv"
    json_file.write_text(json.dumps(ROWS), encoding="utf-8")
    csv_file.write_text(CSV_TEXT, encoding="utf-8")
    expected = clean_and_rank(ROWS, top=3)
    with json_file.open(encoding="utf-8") as jf, csv_file.open(encoding="utf-8") as cf:
        sources: list[object] = [
            json.dumps(ROWS),
            str(json_file),
            json_file,
            str(csv_file),
            csv_file,
            jf,
            cf,
            io.StringIO(CSV_TEXT),
        ]
        for src in sources:
            assert clean_and_rank(src, top=3) == expected


def test_load_list_returns_copy() -> None:
    """Тест: список словників повертається копією."""
    loaded = load_movies(ROWS)
    loaded[0]["title"] = "changed"
    assert ROWS[0]["title"] == " the matrix "


def test_loader_errors(tmp_path: Path) -> None:
    """Тест: непідтримувані джерела."""
    with pytest.raises(TypeError):
        load_movies(42)
    with pytest.raises(ValueError):
        load_movies([1, 2])
    with pytest.raises(ValueError):
        load_movies('{"title": "x"}')
    bad = tmp_path / "movies.txt"
    bad.write_text("x", encoding="utf-8")
    with pytest.raises(ValueError):
        load_movies(bad)


def test_reduce_and_attrgetter() -> None:
    """Тест: total_votes через reduce, titles через attrgetter."""
    records = [normalize(r) for r in ROWS]
    assert total_votes(records) == 85
    assert titles(to_movies(records[:2])) == ["The Matrix", "Inception"]


def test_best_by_genre() -> None:
    """Тест: найкращий фільм кожного жанру."""
    best = best_by_genre(to_movies(map(normalize, ROWS)))
    assert {g: m.title for g, m in best.items()} == {
        "sci-fi": "Inception",
        "drama": "Joker",
        "musical": "Cats",
    }
```

test_lab07.py рядки 177-213

```python
def movie(title: str, rating: float, votes: int) -> Movie:
    """Скорочений конструктор Movie для тестів."""
    return Movie(title=title, genre="drama", year=2020, rating=rating, votes=votes)


def test_total_ordering_all_operators() -> None:
    """Тест: усі оператори порівняння."""
    low, high = movie("A", 7.0, 10), movie("B", 8.0, 5)
    same = movie("A", 7.0, 10)
    assert low < high and low <= high and high > low and high >= low
    assert low == same and low != high and low <= same and low >= same
    assert movie("A", 8.0, 1) < movie("A", 8.0, 2) < movie("B", 8.0, 2)


def test_ordering_with_other_types() -> None:
    """Тест: для чужих типів повертається NotImplemented."""
    m = movie("A", 7.0, 10)
    assert (m == 5) is False
    assert m.__lt__(5) is NotImplemented
    with pytest.raises(TypeError):
        _ = m < 5


def test_sorted_max_and_hash() -> None:
    """Тест: sorted, max та узгоджений хеш."""
    movies = [movie("B", 8.0, 5), movie("A", 7.0, 10), movie("C", 9.0, 1)]
    assert [m.title for m in sorted(movies)] == ["A", "B", "C"]
    assert max(movies).title == "C"
    assert len({movie("A", 7.0, 10), movie("A", 7.0, 10)}) == 1


def test_top_n_helper() -> None:
    """Тест: top_n з від'ємним і великим n."""
    records: list[MovieRecord] = [normalize(r) for r in ROWS]
    assert top_n(-1)(records) == []
    assert len(top_n(100)(records)) == len(records)
    assert list(mapf(lambda r: r["year"])(records[:2])) == [1999, 2010]
```

Запуск програми

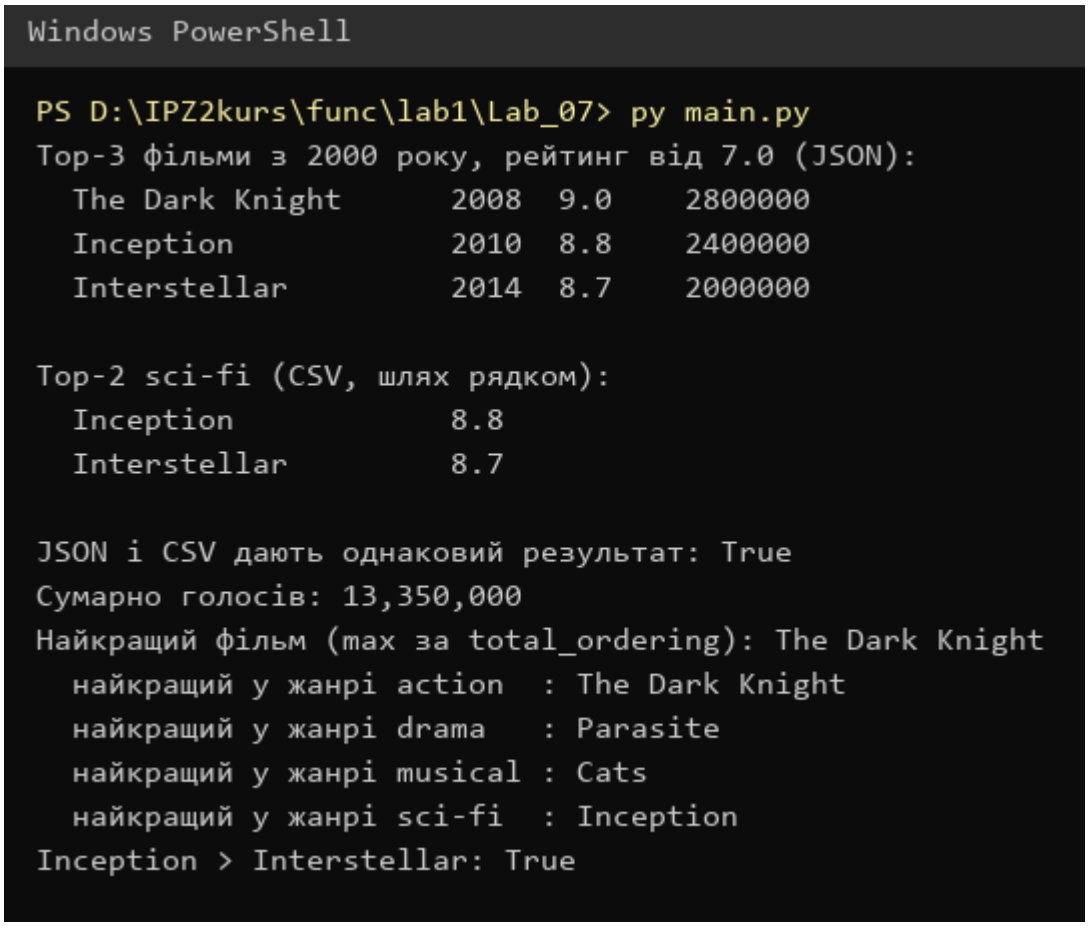


Рис. 1. Вивід програми (python main.py)

Виконання тестів

```
Windows PowerShell

PS D:\IPZ2kurs\func\lab1\Lab_07> py -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.13.9, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\LENOVO\AppData\Local\Programs\Python\Python313\python.exe
rootdir: D:\IPZ2kurs\func\lab1\Lab_07
configfile: pyproject.toml
testpaths: tests
plugins: platformdirs-4.12.4
collecting ... collected 16 items

tests/test_lab07.py::test_pipeline_order_and_empty PASSED                [  6%]
tests/test_lab07.py::test_reduce_initial_value PASSED                    [ 12%]
tests/test_lab07.py::test_partial_filters PASSED                         [ 18%]
tests/test_lab07.py::test_normalize_with_operator PASSED                 [ 25%]
tests/test_lab07.py::test_sort_rank_multi_key PASSED                     [ 31%]
tests/test_lab07.py::test_itemgetter_multiple_keys PASSED                [ 37%]
tests/test_lab07.py::test_clean_and_rank_top PASSED                      [ 43%]
tests/test_lab07.py::test_singledispatch_all_sources PASSED              [ 50%]
tests/test_lab07.py::test_load_list_returns_copy PASSED                  [ 56%]
tests/test_lab07.py::test_loader_errors PASSED                           [ 62%]
tests/test_lab07.py::test_reduce_and_attrgetter PASSED                   [ 68%]
tests/test_lab07.py::test_best_by_genre PASSED                           [ 75%]
tests/test_lab07.py::test_total_ordering_all_operators PASSED            [ 81%]
tests/test_lab07.py::test_ordering_with_other_types PASSED               [ 87%]
tests/test_lab07.py::test_sorted_max_and_hash PASSED                     [ 93%]
tests/test_lab07.py::test_top_n_helper PASSED                            [100%]

============================== 16 passed in 0.10s ==============================
```

Рис. 2. Результат запуску pytest

Перевірка типів і стилю

```
Windows PowerShell

PS D:\IPZ2kurs\func\lab1\Lab_07> py -m mypy .
Success: no issues found in 5 source files
PS D:\IPZ2kurs\func\lab1\Lab_07> py -m ruff check .
All checks passed!
PS D:\IPZ2kurs\func\lab1\Lab_07> py -m black --check .
All done!
5 files would be left unchanged.
```

Рис. 3. mypy, ruff та black без зауважень

Висновок:

Під час виконання роботи побудовано функціональний конвеєр обробки фільмів на partial, reduce та засобах модуля operator майже без явних циклів. Поліморфна функція load_movies на singledispatch однаково працює з JSON, CSV, шляхами, файлами і списками, що підтверджено тестами. Клас Movie з

total_ordering дав повний набір порівнянь за мінімуму коду, що дозволило використовувати max і sorted напряму.